

1-Savol:

1. GUI dasturlashda “hodisalarga asoslangan” (event-driven) yondashuv nima va PyQt5 dagi signal va slot mexanizmi bu yondashuvni amalda qanday ta’minlab beradi?

Javob: GUI (Grafik Foydalanuvchi Interfeysi) dasturlashda **"hodisalarga asoslangan" (event-driven)** yondashuv va PyQt5 dagi **Signal/Slot** mexanizmi bir-biri bilan chambarchas bog‘liq bo‘lgan muhim tushunchalardir. Keling, ularni sodda qilib tushunib olamiz.

1. Hodisalarga asoslangan (Event-driven) yondashuv nima?

An'anaviy (protsedurali) dasturlashda dastur qadam-baqadam o‘z ishini bajaradi va tugaydi. Uning qachon nima qilishini dasturchi yozgan qat’iy ketma-ketlik hal qiladi.

Hodisalarga asoslangan dasturlashda esa dasturning ishlash oqimini foydalanuvchi yoki tizim yaratadigan "hodisalar" (events) boshqaradi. Dastur ishga tushgach, orqa fonda **"Hodisalar sikli" (Event Loop)** deb ataladigan cheksiz sikl aylanib, kutish rejimiga o‘tadi.

Hodisalarga quyidagilar kirishi mumkin:

- Foydalanuvchi sichqoncha bilan tugmani bosishi.
- Klaviaturadan biror harf terilishi.
- Oynaning o‘lchami o‘zgartirilishi.
- Tizimdan kelgan xabar (masalan, taymer tugashi).

Hodisa yuz berganda, Event Loop uni darhol tutib oladi va shu hodisaga biriktirilgan maxsus funksiyani (Event Handler) ishga tushiradi. Funksiya o‘z ishini tugatgach, dastur yana kutish rejimiga qaytadi.

2. PyQt5 dagi Signal va Slot mexanizmi qanday ishlaydi?

PyQt5 (va umuman Qt freymvorki) yuqoridagi "hodisalarni tutib olish va unga javob qaytarish" jarayonini o‘zining mashhur va juda qulay bo‘lgan **Signal va Slot** arxitekturasini orqali amalga oshiradi.

Bu mexanizm uchta asosiy qismdan iborat:

- **Signal (Signal):** Vidjet bilan biror voqea sodir bo‘lganda u tarqatadigan xabar (emit). Masalan: `QPushButton` (tugma) bosilganda `clicked` signalini tarqatadi, matn maydoni o‘zgarganda `textChanged` signalini beradi.

- **Slot (Slot):** Bu ma'lum bir signal tarqatilganda ishga tushishi kerak bo'lgan oddiy Python funksiyasi yoki obykti (metodi). U reaksiyani, ya'ni hodisaga javobni o'zida saqlaydi.
- **Bog'lash (Connection):** Signal tarqatilganda aynan qaysi slot ishga tushishini ta'minlovchi ko'priq. Bu `connect()` metodi orqali amalga oshiriladi.

Amaliy misolda ko'ramiz:

Tasavvur qiling, bizda bitta tugma bor va uni bosganimizda ekranga xabar chiqishi kerak.

```
!pip install pyqt5
```

```
import sys
from PyQt5.QtWidgets import QApplication, QPushButton, QWidget, QVBoxLayout

# 1. SLOT yaratamiz (bu shunchaki voqeaga javob beruvchi funksiya)
def xabar_chiqar():
    print("Tugma muvaffaqiyatli bosildi!")

app = QApplication(sys.argv)
oina = QWidget()
layout = QVBoxLayout()

# Tugma yaratamiz
tugma = QPushButton("Meni bos!")

# 2. SIGNAL va SLOTni bog'laymiz
# tugma.clicked - bu SIGNAL (tugma bosilganda kelib chiqadi)
# .connect(xabar_chiqar) - bu SIGNALni bizning SLOT funksiyamizga ulash
tugma.clicked.connect(xabar_chiqar)

layout.addWidget(tugma)
oina.setLayout(layout)
oina.show()

# 3. Hodisalar siklini (Event Loop) ishga tushiramiz
sys.exit(app.exec_())
```

Bu yondashuvning afzalliklari nimada?

- **Yuqori darajadagi mustaqillik (Loose Coupling):** Signal tarqatayotgan vidjet (masalan, tugma) o'zining signalini kim va qanday qabul qilayotgani bilan ishi yo'q. Slot (funksiya) ham o'zini qaysi vidjet chaqirganini bilishi shart emas. Bu kodni o'zgartirish va kengaytirishni juda osonlashtiradi.

- **Ko'pga-Ko'p (Many-to-Many) munosabatlar:** Bitta signalga bir nechta turli xil slotlarni ulashingiz mumkin (bitta tugmani bossangiz ham oyna yopiladi, ham ma'lumot saqlanadi). Shuningdek, turli xil signallarni bitta yagona slotga ulash ham mumkin.

2-Savol: PyQt5 arxitekturasida vidjetlar o'rtasidagi "Ota-bola" (Parent-Child) bog'liqligi xotirani avtomatik boshqarishda va dizayn ierarxiyasini tuzishda qanday muhim rol o'ynaydi?

Javob: PyQt5 (va umuman Qt freymvorki) arxitekturasining eng asosiy tamoyillaridan biri bu **"Obyektlar daraxti" (Object Tree)** mexanizmidir. Bunda har bir vidjet (element) o'zining "otasi"ga (parent) ega bo'lishi va o'z navbatida boshqa vidjetlarga "ota" bo'lishi mumkin.

Bu "Ota-bola" yondashuvi asosan ikkita juda muhim vazifani hal qilib beradi:

1. Xotirani avtomatik boshqarish (Memory Management)

PyQt5 orqasida C++ tilida yozilgan Qt kutubxonasi yotadi. Odatda, pastki darajadagi tillarda xotirani qo'lda boshqarish, kerak bo'lmagan obyektlarni xotiradan tozalash juda mashaqqatli jarayon. PyQt5 da esa bu "Ota-bola" tizimi orqali juda nafis hal qilingan:

- **Avtomatik tozalash (Garbage Collection muqobili):** Agar siz asosiy oynani (ota vidjetni) yopsangiz yoki xotiradan o'chirsangiz, u o'ziga tegishli bo'lgan barcha "bola" vidjetlarni (ichidagi barcha tugmalar, matnlar, rasmlar va h.k.) xotiradan **avtomatik ravishda o'chiradi**.
- **Xavfsizlik:** Dasturchi har bir tugma yoki yozuv uchun xotirani bo'shatish kodini alohida yozib o'tirishi shart emas. Bu xotira sizib chiqishi (memory leak) xavfini keskin kamaytiradi.

2. Dizayn ierarxiyasi va vizual joylashuv

"Ota-bola" munosabati nafaqat orqa fondagi xotirani, balki vidjetlarning ekranda qanday ko'rinishi va o'zini qanday tutishini ham to'g'ridan-to'g'ri belgilaydi:

- **Chegaralanish (Clipping):** Bola vidjet har doim ota vidjetning ichida (uning chegaralarida) chiziladi. Agar bolaning o'lchami kattalashib ketsa ham, u ota oynasidan tashqariga chiqib keta olmaydi (kesib tashlanadi).
- **Nisbiy koordinatalar:** Bola vidjetning (x, y) koordinatalari kompyuterning butun ekraniga nisbatan emas, balki aynan **ota vidjetning yuqori chap**

burchagiga nisbatan hisoblanadi. Bu oynani siljitganda ichidagi barcha elementlar ham birga siljishini ta'minlaydi.

- **Holatni meros qilib olish:** Agar ota vidjet yashirilsa (`hide()`), uning barcha bolalari ham ko'zdan yo'qoladi. Agar ota oyna nafaol holatga o'tkazilsa (`setEnabled(False)`), uning ichidagi barcha tugmalar va kiritish maydonlari ham bosilmaydigan holatga keladi.

Amaliy misol:

Quyidagi kodda "Ota-bola" bog'liqligi qanday o'rnatilishini ko'rishingiz mumkin:

```
import sys
from PyQt5.QtWidgets import QApplication, QWidget, QPushButton

app = QApplication(sys.argv)

# 1. OTA vidjet (Asosiy oyna) yaratamiz
asosiy_oyna = QWidget()
asosiy_oyna.setWindowTitle("Ota Oyna")
asosiy_oyna.resize(300, 200)

# 2. BOLA vidjet (Tugma) yaratamiz
# E'tibor bering: ikkinchi argument sifatida 'asosiy_oyna' ni beryapmiz.
# Shu orqali tugma o'zining otasi kimligini taniydi.
tugma = QPushButton("Men bola vidjetman", asosiy_oyna)

# Bu koordinatalar (50, 50) kompyuter ekraniga emas,
# balki 'asosiy_oyna'ning ichki qismiga nisbatan olinadi.
tugma.move(50, 50)

asosiy_oyna.show()

# Asosiy oyna yopilganda, 'tugma' ham xotiradan avtomatik o'chadi.
sys.exit(app.exec_())
```

An exception has occurred, use %tb to see the full traceback.

SystemExit: 0

C:\Users\User\AppData\Local\Programs\Python\Python313\Lib\site-packages\IPython\core\interactiveshell.py:3755: UserWarning: To exit: use 'exit', 'quit', or Ctrl-D.

warn("To exit: use 'exit', 'quit', or Ctrl-D.", stacklevel=1)

Xulosa: Parent-Child arxitekturasi PyQt5 dasturlashning "yuragi" hisoblanadi. U GUI (interfeys) qismlarini mantiqiy guruhlariga ajratishga, kodni tartibli saqlashga va eng asosiysi — xotirani xavfsiz va bosh og'riqsiz boshqarishga imkon beradi.

3-savol: Dasturning vizual qismini QtDesigner yordamida alohida yaratish va uni Python mantiqiy kodidan ajratish dastur arxitekturasiga va kelajakda uni rivojlantirishga qanday afzalliklar beradi?

Javob: Dasturning ko'rinishini (UI - User Interface) va uning miyasini (Python mantiqiy kodini) bir-biridan alohida saqlash dasturlash olamidagi eng oltin qoidalaridan biri hisoblanadi. Bu yondashuv arxitekturada **"Separation of Concerns"** (Vazifalarni taqsimlash) yoki **MVC** (Model-View-Controller) tamoyiliga asoslanadi.

Qt Designer yordamida interfeysni alohida `.ui` (XML) faylida yaratish va Python kodiga ulamasdan oldin ajratib qo'yish dasturga kelajakda quyidagi ulkan afzalliklarni beradi:

1. Kodning toza va o'qilishi oson bo'lishi (Clean Code)

Agar interfeysni sof Python kodida yozsangiz, bitta oddiy oyna yaratish uchun yuzlab qator kod yozishga to'g'ri keladi (har bir tugmaning rangi, o'lchami, joylashuvi va h.k.). Natijada dasturning asosiy mantig'i (ma'lumotlar bazasi bilan ishlash, hisob-kitoblar) shu interfeys kodlari orasida yo'qolib ketadi. Qt Designer orqali vizual qism ajratilsa, sizning Python faylingizda faqatgina toza, ixcham va tushunarli mantiqiy jarayonlar qoladi.

2. Dizaynni tezkor yaratish va ko'rish (WYSIWYG)

Qt Designer yordamida siz elementlarni sichqoncha bilan tortib tashlash (drag-and-drop) orqali juda tez joylashtirasiz. Kod yozganda dasturni har safar ishga tushirmaguncha tugma qayerda turganini ko'ra olmaysiz. Designer'da esa oynaning yakuniy holati jarayonning o'zidayoq ko'z oldingizda bo'ladi. Bu dizayn yaratish tezligini o'nlab barobarga oshiradi.

3. Jamoaviy ishlash imkoniyati (Team Collaboration)

Katta loyihalarda odatda dizaynerlar va dasturchilar alohida ishlaydi. Interfeysni ajratish orqali jamoaviy ishni parallel olib borish mumkin:

- **UI/UX mutaxassisi** faqat Qt Designer'da `.ui` fayl bilan ishlaydi va chiroyli oyna yaratadi.
- **Python dasturchisi** esa shu vaqtning o'zida ma'lumotlarni qayta ishlash logikasini yozadi. Ular bir-birining fayliga xalaqit bermaydi va kodda ziddiyatlar (merge conflicts) kelib chiqmaydi.

4. Oson yangilash va xavfsizlik (Maintainability)

Tasavvur qiling, dastur tayyor bo'ldi, lekin buyurtmachi "Kirish" tugmasini chapdan o'ngga olib o'tishni va qizil rangga bo'yashni so'radi. Agar hamma narsa bitta Python faylda yozilgan bo'lsa, siz kodni o'zgartirayotib tasodifan muhim mantiqiy funktsiyani ham buzib qo'yishingiz mumkin. Agar UI alohida bo'lsa, siz shunchaki Qt Designer'ni ochasiz, tugmani surib qo'yasiz va saqlaysiz. Python kodiga umuman tegmaysiz. Bu xatoliklar kelib chiqish xavfini nolga tushiradi.

5. Qayta foydalanish (Reusability)

Yaratilgan bitta `.ui` dizayn faylini turli xil Python skriptlarida qayta-qayta ishlatish mumkin. Masalan, bitta "Sozlamalar" oynasi dizaynini yaratib olib, uni ham mijozlar dasturiga, ham admin paneliga ulash mumkin.

Xulosa: Vizual qismni Qt Designer'da alohida yaratish — bu xuddi bino qurishda uning g'ishtlari va poydevorini (mantiq) bino fasadi va pardozidan (dizayn) alohida qilishdek gap. Bu kelajakda dasturni kengaytirishni, xatolarni topishni va yangi funktsiyalar qo'shishni judayam oson, tartibli va xavfsiz qiladi.

4-savol: Nima uchun GUI oyna elementlarini joylashuvini piksel o'lchamlari bilan belgilashdan ko'ra, turli ekranlarga moslasha oladigan dinamik maketlar (Layouts) dan foydalanish to'g'ri amaliyot hisoblanadi?

Javob: GUI oyna elementlarini (tugmalar, matn maydonlari, rasmlar) mutlaq piksel koordinatalari (`x: 100`, `y: 50`) bilan joylashtirish dastlab oson va tushunarli tuyulishi mumkin. Ammo haqiqiy va professional dasturlarda bu juda katta muammolarni keltirib chiqaradi.

Piksel o'rniga **Dinamik maketlar (Layouts — masalan, `QVBoxLayout`, `QHBoxLayout`, `QGridLayout`)** dan foydalanish "Oltin standart" hisoblanadi. Nima uchun bunday ekanligining 5 ta asosiy sababi bor:

1. Oynani o'lchamini o'zgartirish (Window Resizing)

Eng keng tarqalgan muammo: agar siz elementlarni piksel bilan qotirib qo'ysangiz va foydalanuvchi dastur oynasini kattalashtirsa (Maximize), barcha tugmalar va matnlar yuqori chap burchakda qotib qolaveradi. Oynaning qolgan qismi bo'm-bo'sh, xunuk holatda qoladi. **Layoutlar** esa oynaning yangi o'lchamini sezadi va o'z ichidagi elementlarni shu bo'shliqqa mutanosib ravishda cho'zadi yoki markazlashtiradi.

2. Turli xil ekran rezolyutsiyalari va DPI (Zichlik)

Siz dasturni yozgan kompyuter ekrani bilan foydalanuvchining ekrani bir xil bo'lmaydi.

- Sizning 1080p (FullHD) ekraningizda 150 piksel uzunlikdagi tugma normal ko'rinishi mumkin.
- Lekin foydalanuvchi dasturni zamonaviy 4K (UltraHD) monitorda ochsa, o'sha 150 piksel tugma ko'zga ko'rinmaydigan darajada kichkinalashib ketadi.

Layoutlar elementlarni ekran zichligiga qarab avtomatik masshtablaydi.

3. Matnlarni mahalliyashtirish (Localization / Tarjima)

Tasavvur qiling, siz dastur yozdingiz va "Save" degan tugmaga eni: 50 piksel deb o'lcham berdingiz. Keyinchalik dasturni o'zbek tiliga o'girishdi va yozuv "Saqlash" ga o'zgardi. Rus tilida esa "Сохранить". Pikselda qotirilgan tugmada yangi matnlar sig'may qoladi va yarmi kesilib qoladi (masalan, "Coxp..."). **Layoutlar** esa ichidagi matn uzunligiga qarab tugmaning minimal enini avtomatik ravishda hisoblab, o'zini o'zi kengaytiradi.

4. Operatsion tizimlar va shriftlar farqi (Cross-platform)

Windows, macOS va Linux tizimlari shriftlarni (matnni chizishni) va oyna bezaklarini bir-biridan butunlay farqli usulda amalga oshiradi. Arial 12pt shrifti Mac'da birmuncha joy egallasa, Windows'da boshqacha joy egallashi mumkin. Shuningdek, tizimlarning standart tugma dizaynlari turlicha. Piksel bilan ishlash dasturni bir tizimda chiroyli, ikkinchisida xunuk qilib qo'yadi.

5. Dasturni yangilash va texnik xizmat (Maintainability)

Tasavvur qiling, sizda pastga qarab ketma-ket joylashgan 10 ta matn maydoni (LineEdit) bor. Agar piksel ishlatsangiz, ularning har biriga y=30, y=70, y=110... deb qo'lda koordinata berasiz. Ertaga mijoz 2 va 3-qatorlar orasiga bitta yangi tugma qo'shishni so'rasa, siz yangi tugmadan keyingi **barcha** 8 ta elementning y koordinatasini qo'lda qayta hisoblab chiqishingiz kerak bo'ladi. **Layoutda (QVBoxLayout)** esa shunchaki ro'yxatning kerakli joyiga addWidget() qilasiz, qolgan hamma elementlar avtomatik ravishda pastga suriladi!

5-savol: Nega og'ir hisoblash yoki tarmoq jarayonlari grafik interfeysning qotib qolishiga olib keladi va bu muammoni QThread yordamida, asosiy oqimga (Main Thread) xalaqit bermasdan qanday hal qilish mumkin?

Javob: Bu GUI dasturlashdagi eng ko'p uchraydigan va eng muhim muammolardan biri. Buni to'liq tushunish uchun avvalo "dastur qanday nafas olishi"ni bilishimiz kerak.

1. Nima uchun GUI qotib qoladi? (Muammoning ildizi)

GUI dasturlari (PyQt5, Tkinter va boshqalar) kompyuterda bitta asosiy yo‘lakda — **Asosiy oqimda (Main Thread)** ishlaydi. Bu oqimning ichida dasturni "tirik" ushlab turuvchi **"Hodisalar sikli" (Event Loop)** tinimsiz aylanib turadi.

Event Loop uzluksiz ravishda sichqoncha harakatlarini, tugmalar bosilishini va oynani grafik jihatdan yangilab chizishni (repaint) boshqaradi.

Tasavvur qiling, siz bitta tugmani bosganingizda 10 soniya davom etadigan internetdan katta fayl yuklab olish (network request) yoki 1 millionta qatorni hisoblash funksiyasini yozdingiz. Sizning bu funksiyangiz Event Loop o‘z ishini bajarayotgan aynan o‘sha "Asosiy oqim" ni band qilib qo‘yadi. Dasturingiz o‘sha og‘ir ishni bajarishga sho‘ng‘iydi va **Event Loop 10 soniyaga aylanmay to‘xtab qoladi**. Natijada:

- Sichqonchani qimirlatsangiz ham dastur sezmaydi.
- Boshqa tugmalarni bosa olmaysiz.
- Oynaning rasmi yangilanmaydi.
- Operatsion Tizim dasturingizni **"Javob bermayapti" (Not Responding / He отвечает)** deb o‘ylaydi va uning ustiga oq ekran tortib qo‘yadi.

2. Muammoni QThread bilan qanday hal qilamiz?

Yechim juda oddiy: Asosiy oqimni (Event Loop'ni) faqat va faqat GUI bilan ishlashga (rasm chizish, foydalanuvchini eshitish) qoldirish kerak. Og‘ir vazifani esa **Parallel Oqimga (Background Thread)** o‘tkazish kerak. PyQt5 da bu maqsad uchun **QThread** klassi ishlatiladi.

Bu jarayonni to‘g‘ri amalga oshirishning qat’iy qoidalari bor:

1. **Vazifani ajratish:** Og‘ir hisoblash jarayonini yoki fayl yuklash kodini **QThread** dan meros olgan yangi klassning **run()** metodiga yozasiz.
2. **To‘g‘ridan-to‘g‘ri yozish taqiqlanadi (Oltin qoida):** Orqa fon oqimidan (QThread) turib hech qachon ekrandagi vidjetlarga to‘g‘ridan-to‘g‘ri o‘zgartirish kiritmang (masalan, **self.label.setText()**). GUI faqat Asosiy oqimda yangilanishi shart, aks holda dastur qulaydi (crash).
3. **Aloqa bog‘lash (Signal va Slot):** QThread orqa fonda ishlarni bajarayotganda, o‘zining qancha qismini tugatgani yoki qanday natija olgani haqida faqat **Signal** tarqatishi (**emit()**) kerak. Asosiy oqim bu signalni o‘zining **Slot’iga** ulangan holda kutib oladi va Asosiy oqimning o‘zi ekrandagi yozuvni yoki ProgressBar'ni yangilaydi.

Qisqacha misol:

```
from PyQt5.QtCore import QThread, pyqtSignal
import time
```



```
# 1. Orqa fonda ishlovchi oqim yaratamiz
class OgirIshOqimi(QThread):
    # Bu signal orqali Asosiy oqimga foizlarni yuboramiz
    progress_signal = pyqtSignal(int)

    def run(self):
        for i in range(1, 101):
            time.sleep(0.1) # Qandaydir og'ir hisoblash yoki yuklash jarayoni
            self.progress_signal.emit(i) # Asosiy oqimga xabar beramiz
```

Buni amaliyotda his qilib ko'ramiz

Quyidagi interaktiv vidjetda siz bu muammoni o'z ko'zingiz bilan ko'rishingiz mumkin. Ikkala holatda ham doimiy aylanib turuvchi halqa mavjud (bu sizning interfeysingizning tirik ekanini bildiruvchi Event Loop).

1. **"Asosiy oqimda"** tugmasini bosing va nima bo'lishini ko'ring (Oyna qotadi).
2. **"Parallel oqimda (QThread muqobili)"** tugmasini bosing va interfeys qotmasdan og'ir ish qanday bajarilishini kuzating.

6-savol: Asosiy oyna (QMainWindow), oddiy vidjet (QWidget) va muloqot oynasi (QDialog) sinflarining konseptual farqlari nimalardan iborat va ularning har biri qanday vaziyatlarda ishlatilishi maqsadga muvofiq?

Javob: PyQt5 da oyna yaratish uchun asosan shu uchta sinfning (class) biri tanlanadi. Garchi ularning uchtasi ham ekranda oyna bo'lib ko'rinsa-da, ularning "orqa fon" mantiqi va arxitekturasini butunlay turlicha.

Ularni to'g'ri tanlash dasturning kelajakdagi imkoniyatlarini belgilab beradi. Keling, ularni birma-bir ko'rib chiqamiz:

1. **QWidget** – Poydevor (Bo'sh oyna)

QWidget PyQt5 dagi barcha vizual elementlarning "otasi" hisoblanadi. Tugma (**QPushButton**) ham, yozuv (**QLabel**) ham asli shu sinfdan olingan. Agar siz **QWidget** ni hech qanday "ota" (parent) ko'rsatmasdan yaratsangiz, u kompyuter ekranida **mutlaqo bo'sh, oddiy oyna** bo'lib ochiladi.

- **Xususiyati:** Unda Menyu qo'shadigan, Holat panelini (Status Bar) qo'shadigan tayyor joylar yo'q. U shunchaki to'rtburchak maydon.
- **Qachon ishlatiladi?**

- Dastur juda sodda bo'lsa va faqatgina bir nechta tugma/matndan iborat kichik oyna kerak bo'lsa (masalan, kalkulyator).
- Murakkab dizaynlar yaratishda ichki konteyner (boshqa elementlarni o'ziga yig'uvchi blok) sifatida.
- Noldan o'z shaxsiy vidjetingizni (Custom Widget) yaratayotganingizda.

2. `QMainWindow` – Asosiy Oyna (Karkas)

`QMainWindow` – bu haqiqiy, to'laqonli va yirik dasturlarning "yuzi" uchun mo'ljallangan maxsus sinf. U o'z ichida oldindan tayyorlangan, qat'iy **arxitekturaviy ramkaga (karkasga)** ega.

- **Xususiyati:** Uning ichi bo'sh emas. U 5 ta maxsus hududga bo'lingan:
 1. *Menu Bar (Menyu paneli)* - Yuqoridagi "Fayl", "Tahrirlash", "Yordam" kabi qism.
 2. *Tool Bar (Asboblari paneli)* - Tezkor tugmalar turadigan qator.
 3. *Dock Widgets* - Ekran chekkalariga yopishib turadigan, ajratib olish mumkin bo'lgan yordamchi oynalar (Photoshop'dagi asboblari kabi).
 4. *Status Bar (Holat paneli)* - Eng pastda dastur holatini ko'rsatuvchi yozuv.
 5. **Central Widget (Markaziy vidjet)** - Dasturning asosiy ish joyi (majburiy qism).
- **Qachon ishlatiladi?** Har bir dasturda odatda **faqat bitta** `QMainWindow` bo'ladi. U dastur ochilganda ko'rinadigan birinchi va eng katta oyna bo'lib, barcha jarayonlarni o'zida birlashtiradi (masalan, MS Word, Web Browser yoki Telegram'ning kompyuter versiyasi oynasi).

3. `QDialog` – Muloqot oynasi (Suhbatdosh)

`QDialog` – bu asosiy dastur ishlab turgan vaqtda, foydalanuvchidan qandaydir tasdiq olish yoki unga xabar berish uchun vaqtincha ochiladigan "ikkilamchi" oyna.

- **Xususiyati:** Uning eng katta kuchi — **Modallik (Modality)** xususiyatidir.
 - *Modal oyna:* Agar u ochilsa (masalan, `oyna.exec_()` orqali), uni yopmaguningizcha dasturning orqadagi asosiy oynasiga o'ta olmaysiz va bosa olmaysiz.
 - Unda odatda "OK", "Cancel", "Yes", "No" kabi maxsus natija qaytaruvchi signallar o'rnatilgan bo'ladi.
- **Qachon ishlatiladi?**

- Dasturdan chiqayotganda "Rostdan ham chiqmoqchimisiz?" degan soʻrov oynalarida.
 - Dastur sozlamalari (Settings/Preferences) oynasida.
 - Tizimga kirish (Login/Password) ekranlarida.
 - Faylni yuklash (Open File) muloqot oynalarida.
-

Xulosa (Qisqacha qoida)

- **Yirik dastur yozyapsizmi?** Asosiy oyna uchun darhol `QMainWindow` ishlatilg.
- **Foydalanuvchidan qandaydir ma'lumot yoki qaror soʻramoqchimisiz?** Va u javob bermaguncha dastur kutib turishi kerakmi? Unda `QDialog` ishlatilg.
- **Shunchaki oddiy bir boʻlak, konteyner yoki qandaydir ramkasiz kichik dasturcha kerakmi?** `QWidget` eng toʻgʻri tanlov.

7-savol: PyQt5 da interfeys elementlari dizaynini oʻzgartirish uchun QSS (Qt Style Sheets) dan foydalanish uslubi veb-dasturlashdagi CSS ga qanchalik oʻxshaydi va u kod tozaligiga qanday taʼsir qiladi?

Javob: PyQt5 dagi QSS (Qt Style Sheets) veb-dasturlashdagi CSS (Cascading Style Sheets) ning deyarli toʻliq koʻchirmasi (kopi-pasti) hisoblanadi. Qt yaratuvchilari dasturchilarga qulay boʻlishi uchun ataylab veb-dasturlashdagi eng ommabop va mukammal dizayn tilini ish stoli (desktop) dasturlashiga moslashtirishgan.

1. QSS va CSS qanchalik oʻxshaydi?

Ularning sintaksisi, ishlash mantiqi va hatto asosiy qoidalari 90% bir xil:

- **Sintaksis (Qoidalar yozilishi):** Ikkisida ham `Selektor { xususiyat: qiymat; }` qoidasi ishlaydi. *Vebdagi CSS misoli:* `button { background-color: red; color: white; }` *PyQt5 dagi QSS misoli:* `QPushButton { background-color: red; color: white; }`
- **Psevdo-sinflar (Pseudo-classes):** Tugma ustiga sichqoncha borganda, bosilganda yoki nafaol holatda dizaynni oʻzgartirish QSS da ham aynan vebdagidek: `:hover`, `:pressed`, `:disabled`, `:checked` kabi yoziladi.
- **Quti modeli (Box Model):** Veb-dasturlash dizaynining yuragi boʻlgan `margin` (tashqi boʻshliq), `padding` (ichki boʻshliq) va `border` (chegara) tushunchalari QSS da ham huddi shunday ishlaydi va oʻlchanadi.
- **ID va Sinflar orqali murojaat (Selectors):**

- Vebda bitta aniq elementga ta'sir qilish uchun `id="myButton"` beriladi va `#myButton` deb chaqiriladi. PyQt5 da esa bu `objectName="myButton"` deb beriladi va QSS da yana `#myButton` deb o'zgartiriladi.
- Vebda `class="btn"` ishlatilsa, PyQt5 da to'g'ridan-to'g'ri vidjet sinfi nomidan (`QLineEdit`, `QComboBox`) foydalaniladi.

Yagona farqi: QSS da faqatgina Qt ning o'ziga xos vidjetlari (`div` o'rniga `QWidget` va h.k.) hamda ba'zi maxsus xususiyatlar (masalan, rasmlar joylashuvi uchun `image:` o'rniga `qproperty-icon:`) ishlatiladi.

2. QSS kod tozaligiga qanday ta'sir qiladi?

QSS dan foydalanish dastur arxitekturasini qorishiq koddan (spaghetti code) ideal va professional holatga olib chiqadi:

1. **Vazifalarni taqsimlash (Separation of Concerns):** Agar siz QSS ishlatmasangiz, har bir tugma, matn va oynaning rangini, shriftini to'g'ridan-to'g'ri Python da yozishingizga to'g'ri keladi (`button.setStyleSheet("...")`, `label.setFont(...)`). Natijada dasturning ishlash mantiqi o'nlab qator xunuk dizayn kodlari orasida ko'milib ketadi.
Yechim: Barcha vizual dizayn qoidalari bitta alohida `style.qss` fayliga yoziladi. Python fayllari ichida esa faqat va faqat dastur mantig'i (logika) qoladi. Kod qisqa, toza va o'qilishi oson bo'ladi.
2. **Markazlashgan boshqaruv:** Dasturingizda 50 ta tugma bor deylik. Ularning hammasini qizil rangdan ko'k rangga o'tkazmoqchisiz. Agar kodda yozgan bo'lsangiz, 50 ta joyga kirib o'zgartirib chiqasiz. QSS bilan esa bitta `.qss` faylini topib, faqatgina `QPushButton { background-color: blue; }` qatorini o'zgartirib qo'yasiz. Bir soniyada butun dastur yangilanadi. Bu `QApplication` darajasida ulanganligi sababli barcha oyna va elementlarga birdaniga ta'sir qiladi.
3. **Mavzularni (Themes) oson almashtirish:** Zamonaviy dasturlarda "Kunduzgi rejim" (Light Mode) va "Tungi rejim" (Dark Mode) bo'lishi odatiy hol. Agar buni Python kod bilan qilsangiz, miyangiz qotib ketadi. QSS orqali esa sizda ikkita fayl bo'ladi: `light.qss` va `dark.qss`. Tugma bosilganda Python orqali `app.setStyleSheet(open('dark.qss').read())` deb chaqirib qo'yasiz — tamom, butun dastur qora mavzuga o'tadi.

Xulosa: PyQt5 da QSS dan foydalanish – veb-dasturlashdagi eng zo'r amaliyotlarni ish stoli (desktop) dasturlashga joriy qilish deganidir. U sizga ham tezlik, ham go'zallik, ham

kelajakda bosh og'riqsiz qo'llab-quvvatlash (maintainability) imkonini beradi.

8-savol: Katta hajmdagi ma'lumotlarni (masalan, ro'yxatlar yoki jadvallar) ekranda ko'rsatishda PyQt5 dagi Model-View arxitekturasida oddiy vidjetlarga nisbatan qanday unumdorlik taqdim etadi?

Javob: PyQt5 da ma'lumotlarni jadvallar yoki ro'yxatlarda ko'rsatishning ikkita asosiy usuli bor: **Oddiy vidjetlar** (Item-based: `QTableWidget`, `QListWidget`) va **Model-View arxitekturasida** (`QTableView`, `QListView` va o'ziga xos Model).

Kichik ma'lumotlar uchun oddiy vidjetlar qulay bo'lsa-da, katta hajmdagi ma'lumotlar (masalan, 10 000 yoki 1 million qator) bilan ishlaganda oddiy vidjetlar dasturni "o'ldiradi". Shu o'rinda Model-View arxitekturasida misli ko'rilmagan unumdorlik (performance) taqdim etadi.

Buning 4 ta asosiy sababi bor:

1. Xotirani keskin tejash (Memory Efficiency)

- **Oddiy vidjetlarda:** Jadvaldagi har bir katakcha uchun alohida ob'ekt (`QTableWidgetItem`) yaratiladi. Agar sizda 10 000 qator va 10 ta ustun bo'lsa, xotirada jismonan 100 000 ta interfeys elementi yaratiladi. Bu operativ xotirani (RAM) to'ldirib yuboradi va dasturni qotirib qo'yadi.
- **Model-View tizimida:** Ma'lumotlar oddiy ro'yxat (Python list) yoki ma'lumotlar bazasida (SQL) qanday bo'lsa, shunday xom holicha saqlanadi. Jadval (View) o'zida hech qanday obyekt saqlamaydi, u faqatgina markaziy Model orqali ma'lumotni o'qiydi. Natijada xotira sarfi minimal darajaga tushadi.

2. Faqat ko'rinadigan qismni ishlash (Lazy Rendering / Delegation)

Tasavvur qiling, sizning jadvalingizda 1 million qator bor, lekin foydalanuvchi monitorida bir vaqtning o'zida atigi 50 ta qator ko'rinadi.

- Model-View tizimidagi `QTableView` aqlli ishlaydi: u jami 1 million qatorni emas, faqatgina **ekranda ko'rinib turgan 50 ta qator uchungina** Model'dan ma'lumot so'raydi va faqat shularni chizadi (render qiladi).
- Sichqoncha bilan pastga skroll qilganingizda (aylantirganingizda), u tepadagi o'tib ketgan kataklarni vizual xotiradan tozalaydi va yangi paydo bo'layotganlarini tezkorlik bilan chizib boraveradi. Shuning uchun ham dastur 1 million qatorni ham, 100 ta qatorni ham bir xil tezlikda ochadi.

3. Ma'lumotlar va Dizaynni ajratish (Separation of Concerns)

Oddiy vidjetlarda ma'lumotni o'zgartirish uchun dasturchi har bir katakchani topib, alohida yangilab chiqishi kerak. Model-View yondashuvida esa vizual ko'rinish va ma'lumot bir-biridan mustaqil. Agar orqa fondagi ma'lumotlar bazasida biror o'zgarish bo'lsa, faqatgina Model yangilanadi, View (ekrandagi jadval) esa o'z-o'zidan avtomatik tarzda o'zgaradi.

4. Yashin tezligida saralash va qidirish (Filtering & Sorting)

PyQt5 da `QSortFilterProxyModel` degan maxsus, o'ta kuchli vosita bor. U Model va View orasiga ko'prikk bo'lib o'rnatiladi. Katta jadvallardan biror ismni qidirish yoki summani o'sish tartibida saralash kerak bo'lsa, u og'ir interfeys elementlarini joyidan qo'zg'atmaydi, balki to'g'ridan-to'g'ri orqa fondagi xom ma'lumotlar ro'yxatini saralaydi. View esa bu tayyor natijani zumda ekranga ko'rsatadi.

Xulosa: Agar dasturingizda atigi 100-200 qatorgacha bo'lgan kichik ma'lumot bo'lsa, kodni qisqa qilish uchun oddiy `QTableWidget` ishlatavering. Ammo minglab qatorlik katta MySQL/SQLite bazasi, API dan kelayotgan ulkan JSON ma'lumotlari yoki og'ir log fayllarni ekranga chiqarish kerak bo'lsa, **Model-View** (`QTableView` + `QAbstractTableModel`) dan foydalanish yagona to'g'ri amaliyot hisoblanadi. Bu dasturning tez ishlashini va foydalanuvchiga muammosiz xizmat qilishini ta'minlaydi.

9-savol: PyQt5 da foydalanuvchi harakatlarini tutib olishda standart "signal-slot" mexanizmidan foydalanish va bevosita klaviatura yoki sichqoncha hodisalarini (masalan, `KeyPressEvent`) qayta ishlash usullarining qanday farqlari bor?

Javob: Bu savol aslidagi oldingi xabarlarining birida tushib qolgan edi, endi buni to'g'rilab, **9-savol** sifatida batafsil javob beraman.

PyQt5 da foydalanuvchi harakatlarini boshqarishning bu ikkala usuli ham o'ta muhim, lekin ular **turli darajadagi abstraksiyalarda (chuqurlikda)** ishlaydi. Eng sodda qilib aytganda: "Signal va Slot" bu qayta ishlangan tayyor **xulosa** bo'lsa, "Hodisa (Event)ni tutib olish" bu eng quyi darajadagi **xom ma'lumotdir**.

Ularning o'rtasidagi asosiy farqlarni ko'rib chiqamiz:

1. Signal va Slot mexanizmi (Yuqori daraja - High Level)

Bu PyQt5 ning eng mashhur va qulay usuli. U "Nima mantiqiy ish bajarildi?" degan savolga tayyor javob beradi.

- **Qanday ishlaydi?** Sizning `QPushButton` (tugma) vidjetingiz orqa fonda juda ko'p xom hodisalarni qabul qiladi (sichqoncha bosildi, qo'yib yuborildi, tugma

ustidan ketdi). Vidjet bu xom ma'lumotlarni o'zi tahlil qiladi va faqatgina bitta aniq signal: `clicked` (bosildi) deb xabar tarqatadi.

- **Afzalligi:** Kod yozish juda oson. Vidjetni o'zgartirish (klassdan meros olish) shart emas, shunchaki qatorga `.connect()` yozib qo'ysangiz kifoya. U umumiy mantiqqa e'tibor qaratadi (masalan, oyna kim tomondandir yopildi yoki matn o'zgardi).
- **Qachon ishlatiladi?** Dasturning 90% holatlarida. Standart UI elementlari (tugmalar, chekbokslar, ro'yxatlar) bilan ishlaganda.

2. Hodisalarni qayta ishlash (Event Overriding - Quyi daraja)

Bu usul operatsion tizimdan kelayotgan eng "xom" (raw) ma'lumotni tutib olish imkonini beradi. U "Aynan qanday fizika sodir bo'ldi?" degan savolga javob beradi.

- **Qanday ishlaydi?** Buning uchun siz standart vidjetdan meros olib (subclassing), uning maxsus funksiyalarini (masalan, `mousePressEvent`, `keyPressEvent`, `wheelEvent`) qaytadan yozib chiqishingiz kerak bo'ladi.
- **Afzalligi:** To'liq nazorat va aniqlik. Siz nafaqat sichqoncha bosilganini, balki uning ekrandagi aynan qaysi `(x, y)` koordinatada bosilganini, bosilganda klaviaturadagi `Ctrl` yoki `Shift` tugmalari birga ushlab turilganmi yoki yo'qligini, sichqonchaning o'ng yoki chap tomoni bosilganini bilib ola olasiz.
- **Qachon ishlatiladi?** Standart signallar yetarli bo'lmaganda. Masalan:
 - Rasm chizish dasturida kursorning aniq koordinatalarini kuzatish uchun (`mouseMoveEvent`).
 - Tugmani o'ng bosganda maxsus menyu ochish uchun (`contextMenuEvent`).
 - Drag-and-Drop (fayllarni tortib tashlash) tizimini yaratish uchun (`dragEnterEvent`, `dropEvent`).
 - Noldan boshlab o'z shaxsiy dizayniga ega bo'lgan yangi vidjet yaratganda.

Qisqacha taqqoslash

Xususiyat	Signal va Slot	Event Overriding (Hodisalarni tutish)
Darajasi	Yuqori (Abstrakt, Mantiqiy xulosa)	Quyi (Fizik, Aniq detallar)

Xususiyat	Signal va Slot	Event Overriding (Hodisalarni tutish)
Kod yozish	Oson (1 qator <code>connect</code> kodi)	Murakkab (Klassdan meros olish kerak)
Ma'lumot	Kamroq (Shunchaki "Bosildi")	To'liq (Koordinata, klavish, tugma turi)
Qayerda yoziladi?	Asosiy oynada (Controller)	Vidjetning o'zida (Class ichida)

10-savol: Python va PyQt5 asosida yozilgan dasturni dasturlash tili o'rnatilmagan kompyuterlarda ham ishlaydigan tayyor .exe formatiga o'tkazishda odatda qanday qiyinchiliklar yuzaga keladi va bu jarayon nima uchun kerak?

Javob: Python tilida yozilgan loyihani (ayniqsa GUI dasturlarni) yagona ishga tushuvchi .exe (Windows), .app (macOS) yoki ishga tushuvchi faylga (Linux) aylantirish jarayoni dasturlash tilida "**Kompilyatsiya**" yoki "**Freezing**" (**Muzlatish**) deb ataladi. Bu maqsadda asosan PyInstaller, cx_Freeze yoki Auto-py-to-exe kabi vositalardan foydalaniladi.

Keling, avval bu jarayon nima uchun zarurligini, so'ngra qanday "bosh og'riqlar" keltirib chiqarishini ko'rib chiqamiz.

1. Bu jarayon nima uchun kerak? (Asosiy sabablar)

- **Foydalanuvchi qulayligi (Eng asosiy sabab):** Oddiy kompyuter foydalanuvchisi "Python nima ekanligini" bilmaydi. U qora ekranga (Terminalga) kirib `pip install PyQt5` deb yozishni, qandaydir qaramliklarni (dependencies) o'rnatishni xohlamaydi. U shunchaki belgi (ikonka) ustiga ikki marta bosib dasturni ochishni istaydi. .exe fayl bunga to'liq imkon beradi.
- **Mustaqillik (Standalone):** Dastur .exe qilinganda, u o'zining ichiga butun boshli Python interpretatorini va kerakli barcha kutubxonalarni (jumladan PyQt5 ni ham) yashirib oladi. Mijozning kompyuterida Python umuman o'rnatilmagan bo'lsa ham dastur bimalol ishlayveradi.
- **Kodni himoyalash:** Garchi Python kodini .exe dan qayta o'qish (decompile) qiyin bo'lmasa-da, bu jarayon sizning xom (raw) kodingizni oddiy ko'zdan va

tasodifiy o'zgarishlardan himoya qiladi. Dastur fayllari yaxlit bir blokga aylanadi.

2. Odatda qanday qiyinchiliklar yuzaga keladi?

Dasturni `.exe` holatiga o'tkazish tajribasiz dasturchilar uchun haqiqiy "dahshat"ga aylanishi mumkin. Quyida eng ko'p uchraydigan 4 ta katta muammo keltirilgan:

A. Fayl hajmining keskin kattalashishi

Siz yozgan `.py` skript atigi 50 Kilobayt bo'lishi mumkin. Lekin uni `PyInstaller` orqali `.exe` qilsangiz, hajmi birdaniga 50-80 Megabaytga aylanadi. **Sababi:** PyQt5 juda katta kutubxona (C++ da yozilgan Qt dvigateli). PyInstaller dastur ishlashi uchun butun Qt tizimini va Python interpretatorini bitta `.exe` ichiga majburan qamab qo'yadi.

B. Resurs fayllarning (Rasm, QSS, Baza) yo'qolishi

Bu eng ko'p uchraydigan xatolik. Dasturni PyCharm'da ishga tushsangiz rasmlar ko'rinadi, lekin `.exe` qilsangiz rasmlar yo'qoladi yoki dastur xatolik berib yopilib qoladi. **Sababi:** Yagona `.exe` fayl ishga tushganida, o'zining ichidagi narsalarni vaqtincha kompyuterning `Temp` papkasiga (odatda `_MEIPASS` deb ataladi) yoyadi. Dasturingiz ichidagi kod esa rasmlarni o'sha eski papkadan qidiradi. **Yechimi:** Bunga yo'l qo'ymaslik uchun maxsus `resource.qrc` fayllaridan foydalanish yoki kod ichida fayllar yo'lini to'g'rilovchi maxsus `os.path` funksiyalarini yozish talab etiladi.

C. Antiviruslarning noto'g'ri ishlashi (False Positives)

Siz qanchalik toza va beziyon dastur yozmang, Windows Defender yoki boshqa antiviruslar sizning tayyor `.exe` faylingizni o'chirib yuborishi yoki "Troj/Malware" deb xavf signalini chalishi ehtimoli juda yuqori. **Sababi:** Xakerlar ham viruslarni ko'pincha Python'da yozib, xuddi siz kabi `PyInstaller` yordamida `.exe` ga o'tkazishadi. Antiviruslar `PyInstaller`'ning maxsus paketlash algoritmini taniydi va ehtiyot shart uni bloklaydi.

D. Yashirin qaramliklar (Hidden Imports)

PyQt5 da ba'zi narsalar to'g'ridan-to'g'ri kodda emas, balki orqa fonda, dinamik tarzda chaqiriladi (masalan, ma'lumotlar bazasi drayverlari, pluginlar). Kompilyator `.exe` yig'ayotganda bu yashirin kutubxonalarni ko'rmay qoladi va tayyor dasturga qo'shmaydi. Natijada dastur ishga tushganda "ModuleNotFoundError" xatosi chiqadi.

11-savol: Tarmoq dasturlashda ma'lumotlarni uzatishning TCP va UDP protokollari qanday fundamental prinsiplarga asoslanadi va har bir protokol qanday real loyihalarda tanlanishi kerak?

Javob: Internet tarmog'ida ma'lumotlar (paketlar) asosan ikkita fundamental transport protokoli orqali uzatiladi: **TCP** (Transmission Control Protocol) va **UDP** (User Datagram Protocol). Garchi ikkalasi ham ma'lumotni A nuqtadan B nuqtaga yetkazishga xizmat qilsada, ularning ishlash falsafasi bir-biriga butunlay qarama-qarshidir.

Buni tushunish uchun ularni oddiy hayotdagi aloqa vositalariga qiyoslaymiz.

1. TCP (Transmission Control Protocol) – "Ishonchli Telefon Qo'ng'irog'i"

Fundamental prinsiplari: TCP ning asosiy maqsadi — **Ishonchlilik (Reliability)**. U ma'lumotning biror bayti ham yo'qolmasligini va aynan jo'natilgan ketma-ketlikda yetib borishini 100% kafolatlaydi.

- **Aloqa o'rnatish (Connection-oriented):** Ma'lumot yuborishdan oldin TCP "Uch tomonlama kelishuv" (Three-way handshake) o'tkazadi. Ya'ni: *"Salom, eshityapsanmi?" -> "Ha, eshityapman, o'zing qaleysan?" -> "Zo'r, hozir ma'lumot yuborishni boshlayman!"*.
- **Tasdiqlash (Acknowledgment - ACK):** TCP da jo'natilgan har bir paket uchun qabul qiluvchidan tasdiq javobi kutiladi. *"1-paketni oldingmi? Ha. Endi 2-paketni yuboraman."*
- **Qayta jo'natish (Retransmission):** Agar tarmoqda uzilish bo'lib, qaysidir paket yo'qolib qolsa (tasdiq kelmasa), TCP uzoqroq kutadi va o'sha yo'qolgan paketni topib, yana qaytadan yuboradi.

Qanday loyihalarda tanlanadi? Aniqlik tezlikdan muhimroq bo'lgan har qanday loyihada:

- **Veb-saytlar (HTTP/HTTPS):** Saytdagi bitta harf yoki kod tushib qolsa, sayt ishlamay qoladi.
- **Fayl yuklash (FTP):** Rasm yoki videoning o'rtasidan 1 Kilobayt yo'qolsa ham fayl buziladi (corrupted).
- **Elektron pochta (SMTP/IMAP) va Chatlar:** Xabar to'liq va o'zgarmas yetib borishi shart.
- **Ma'lumotlar bazasi bilan ishlash:** Tranzaksiyalar va SQL so'rovlarda.

2. UDP (User Datagram Protocol) – "Tezkor Pochta Kaptari" (yoki Karnay)

Fundamental prinsiplari: UDP ning asosiy maqsadi — **Tezlik (Speed)**. U ma'lumotni yetib borgan-bormaganligiga umuman qiziqmaydi, tasdiq so'ramaydi va yo'qolgan paketlarni qayta jo'natmaydi.

- **Aloqasiz (Connectionless):** Oldindan hech qanday kelishuv o'tkazilmaydi. Shunchaki manzilni yozadi-yu, ma'lumotni to'xtovsiz otaveradi (Fire-and-forget).
- **Kam xarajat (Low overhead):** Tasdiqlash, tartibga solish va xatolarni tekshirish kabi mexanizmlar yo'qligi sababli, UDP paketlari juda yengil bo'ladi va protsessorni charchatmaydi.

Qanday loyihalarda tanlanadi? Ma'lumotlarning aniqligidan ko'ra ularning real vaqtda (Real-time), tez yetib kelishi muhim bo'lgan loyihalarda (yo'qolgan paketlarga ahamiyat berilmaydi):

- **Onlayn o'yinlar (CS:GO, PUBG):** o'yinchining qayerda turgani har millisoniyada yangilanadi. Agar bitta paket yo'qolsa, uni qayta so'rashning keragi yo'q, chunki o'yinchi allaqachon boshqa joyga o'tib ketgan. Eskirgan ma'lumot kerak emas.
- **Jonli video va audio (Zoom, YouTube Live, Telegram Calls):** Internet qotib qolganda video biroz xiralashadi yoki ovoz uziladi (bu paket yo'qolganining belgisi). Lekin qo'ng'iroq to'xtab qolmaydi, jonli efir davom etaveradi. Agar bu TCP da bo'lganda, yo'qolgan paket yetib kelgunicha ekraningiz 5-10 soniya "Loading" bo'lib qotib turar edi.
- **IoT (Aqlli qurilmalar):** Har soniyada temperaturani serverga jo'natib turuvchi sensorlar (bitta qiymat yetib bormasa, keyingi soniyada yangisi boraveradi).

Qisqacha Taqqoslash Jadvali

Xususiyat	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Maqsad	100% Ishonchlilik va Kafolat	Maksimal Tezlik va Jonli efir
Tasdiqlash (ACK)	Har bir paket tasdiqlanadi	Tasdiqlanmaydi (Umuman)
Xatolarni tuzatish	Yo'qolgan paket qayta yuboriladi	Yo'qolgan paket tashlab yuboriladi
Tezlik	Sekinroq (Qo'shimcha mexanizmlar hisobiga)	Juda tez

Xususiyat	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Misol	Web (HTTP), Fayl (FTP), E-mail	Jonli Video, Onlayn o'yin, Ovoz

12-savol: Mijoz-server (Client-Server) arxitekturasida serverning asosiy mas'uliyati nimalardan iborat bo'ladi va bitta to'liq tarmoq aloqasi sikli qanday bosqichlardan o'tadi?

Javob: Mijoz-server (Client-Server) arxitekturasida butun zamonaviy internetning (veb-saytlar, mobil ilovalar, o'yinlar) asosiy poydevori hisoblanadi. Unda rollar qat'iy taqsimlangan: **Mijoz** (xizmat so'rovchi) va **Server** (xizmat ko'rsatuvchi).

1. Serverning asosiy mas'uliyatlari nimalardan iborat?

Server bu — oddiy qilib aytganda, 24/7 ishlab turuvchi, yuqori quvvatli kompyuter va unda yozilgan dastur. Uning bo'yida quyidagi o'ta muhim mas'uliyatlar bor:

- **Tinglash va Kutish (Listening):** Server doimiy ravishda ma'lum bir portda (masalan, veb-saytlar uchun 80 yoki 443-port) mijozlardan keladigan so'rovlarni quloq tutib kutib turadi.
- **So'rovlarni tahlil qilish (Parsing/Routing):** Mijozdan xat kelganda, server u nima so'rayotganini tahlil qiladi. "Bu foydalanuvchi rasmni ochmoqchimi yoki akkauntiga kirmoqchimi?" degan savolga manzilni (URL) o'qib javob topadi.
- **Mantiqiy amallar va Baza bilan ishlash (Business Logic & Database):** Agar foydalanuvchi "Parolim to'g'rimi?" deb so'rasa, server o'zining Ma'lumotlar bazasiga (Database) kirib, parolni tekshiradi, hisob-kitoblarni amalga oshiradi.
- **Xavfsizlik va Ruxsat (Authentication/Authorization):** So'rov yuborayotgan mijoz haqiqatan ham tizimga kirganmi? Uning bu ma'lumotni o'qishga huquqi bormi? Server har bir qadamda shu xavfsizlikni tekshiradi.
- **Javob shakllantirish (Response generation):** Barcha jarayon tugagach, server mijoz tushunadigan tilda (HTML, JSON, XML yoki fayl ko'rinishida) tayyor javobni paketlab, ortga yuboradi.

2. Bitta to'liq tarmoq aloqasi sikli qanday bosqichlardan o'tadi?

Tasavvur qiling, brauzeringizda `google.com` deb yozib, Enter tugmasini bosdingiz. Shu soniyaning o'zida quyidagi **to'liq sikl (Request-Response Cycle)** sodir bo'ladi:

1. **Mijoz so‘rov yaratishi:** Brauzer nima kerakligini belgilab, xat (HTTP Request) yozishni boshlaydi.
 2. **Manzilni aniqlash (DNS Lookup):** Kompyuter `google.com` ning IP manzilini (masalan, `142.250.190.46`) topish uchun DNS serverga murojaat qiladi (xuddi telefon kitobchasidan raqam qidirgandek).
 3. **Aloqa o‘rnatish (TCP Handshake):** Mijoz va Server o‘rtasida ishonchli ko‘prik quriladi. (Oldingi savoldagi kabi: "Eshityapsanmi? -> Ha -> Gapirdim!").
 4. **So‘rovni jo‘natish (Request Transmission):** Yozilgan xat Internet tarmoqlari, kabellar va ruterlar orqali servergacha yetib boradi.
 5. **Serverda qayta ishlash (Server Processing):** Server xatni ochadi, bazaga qaraydi, kerakli HTML sahifani va rasmlarni tayyorlaydi.
 6. **Javobni qaytarish (Response Transmission):** Server tayyor ma'lumotni Internet orqali mijozga qaytarib yuboradi. Odatda bu yerda "200 OK" (Hammasi zo‘r) yoki "404 Not Found" (Topilmadi) kabi kodlar qo‘shib yuboriladi.
 7. **Mijozda chizish (Rendering):** Brauzer kelgan kodni o‘qiydi va ekraningizda chiroyli tugmalar, matnlar va rasmlarni chizib beradi.
 8. **Aloqani uzish (Connection Close):** Barcha ma'lumot olinib bo‘lgach, TCP ko‘prigi yopiladi (yoki keyingi so‘rovlar uchun biroz kutib turadi).
-

13-savol: Operatsion tizim darajasida “soket” (socket) tushunchasi o‘zi nima va Python dagi socket kutubxonasi yozgan dasturimiz bilan kompyuter tarmog‘i o‘rtasida qanday qilib muloqot ko‘prigini o‘rnatib beradi?

Javob: "Soket" (inglizchada *socket* — rozetka, uya) kompyuter tarmoqlari va internetning eng quyi, poydevor g‘ishtidir. Biz har kuni ishlatadigan brauzerlar, Telegram, onlayn o‘yinlar — bularning barchasi orqa fonda aynan soketlar orqali ishlaydi.

Keling, bu tushunchani operatsion tizim (OS) darajasida va Python misolida ajratib tushunamiz.

1. Operatsion tizim darajasida "Soket" o‘zi nima?

Operatsion tizimlar (Windows, Linux, macOS) kompyuterning jismoniy tarmoq kartasini (Wi-Fi, LAN kabel) to‘g‘ridan-to‘g‘ri boshqaradi. Oddiy dasturlar u yerga bevosita aralasha olmaydi.

Soket — bu Operatsion Tizim tomonidan dasturlarga taqdim etiladigan **dasturiy eshik** yoki **aloqa nuqtasi (endpoint)**.

- **Uy va Eshik o‘xshatishi:** Agar kompyuterning **IP manzili** ko‘p qavatli binoning umumiy manzili bo‘lsa, **Port** (masalan, 80 yoki 443) o‘sha binodagi aniq bir xonaning (dasturning) eshigidir. **Soket** esa o‘sha eshikning qulfi va u orqali o‘tkazilgan aloqa trubasidir.
- **Fayl falsafasi:** Ayniqsa Linux tizimlarida "hamma narsa fayl" tamoyili ishlaydi. Tizim uchun internetdan kelayotgan ma'lumotni qabul qilish yoki jo‘natish — bu shunchaki bitta maxsus faylga (soketga) matn yozish (`write`) va undan o‘qish (`read`) bilan barobar. Ichki murakkab jarayonlarni (paketlarni TCP ga o‘rash, xatolarni tekshirish) Operatsion Tizimning o‘zi (Network Stack) bo‘yniga oladi.

2. Python dagi `socket` kutubxonasi qanday ko‘prik o‘rnatadi?

Python dagi `socket` moduli — bu OS ichida C tilida yozilgan, tushunish juda qiyin bo‘lgan pastki darajadagi API (Berkeley Sockets) buyruqlarini Python'ning qulay, insonbop obyektlariga aylantirib beruvchi **tarjimon**.

Siz Python'da bir necha qator kod yozganingizda, u orqa fonda OS bilan qanday gaplashishini bosqichma-bosqich ko‘ramiz:

1. `s = socket.socket()` (**Yaratish**): Python OS ga: *"Menga internetga chiqish uchun bitta bo‘sh truba (soket) yasab ber"* deydi. OS unga maxsus ID (fayl deskriptor) ajratib, trubaning bir uchini Pythonga beradi.
2. `s.bind(('IP', 8080))` (**Bog‘lash**): Python OS ga: *"Men 8080-raqamli portni (eshikni) band qildim. Shu eshikka kim xat tashlasa, boshqa dasturlarga bermay, to‘g‘ridan-to‘g‘ri mening trubamga (soketimga) tashlaysan"* deydi.
3. `s.listen()` (**Quloq tutish**): Python eshikni ochib qo‘yadi va OS ga: *"Men mijozlarni kutishni boshladim, kimdir ulanmoqchi bo‘lsa menga xabar ber"* deb navbatchilikka o‘tadi.
4. `s.accept()` (**Qabul qilish**): Tashqaridan kimdir ulanganda, OS darchani ochadi. Bu funksiya orqali Python o‘sha aniq mijoz bilan gaplashish uchun maxsus, yakkama-yakka yangi soket (liniya) yaratib oladi.
5. `s.send()` va `s.recv()` (**Ma'lumot almashish**): Endi eng qiziq joyi: Siz Python'da shunchaki `s.send(b"Salom")` deb yozasiz. Python bu 5 ta baytni OS ga beradi. OS esa uni o‘zining tarmog‘i orqali qadoqlab (TCP paketlar), internet kiygizib, dunyoning narigi chekkasidagi kompyuterga uchirib yuboradi.

Xulosa: Soket — bu kompyuteringiz tashqi dunyo bilan gaplashadigan "og‘iz va quloq" bo‘lsa, Python dagi `socket` kutubxonasi OS ning miyasiga to‘g‘ridan-to‘g‘ri buyruq beruvchi "asab tolasi" hisoblanadi. U tarmoqdagi eng og‘ir pastki daraja (low-level)

amallarini yashirib, dasturchiga faqat ma'lumotni o'qish va yozishdek oson vazifani qoldiradi.

14-savol: Nima uchun tarmoq orqali xabar jo'natishda Python obyektlari yoki matnlarini to'g'ridan-to'g'ri yuborib bo'lmaydi va ma'lumotlarni baytlarga o'girish (encode) nega albatta talab etiladi?

Javob: Kompyuterlararo tarmoq muloqotida (socketlar orqali) ma'lumotlarni bevosita Python matni yoki obyekti (masalan, `list` yoki `dict`) ko'rinishida yuborishning iloji yo'q. Buning o'ziga xos juda muhim fundamental va fizik sabablari bor.

Bu holatni tushunish uchun **Lego qasri o'xshatishidan** foydalanamiz. Siz do'stingizga o'zingiz qurgan ulkan Lego qasrini (Python obyekti) kichkina quvur (Tarmoq kabeli) orqali yubormoqchisiz. Qasrni butunligicha quvurga tiqib bo'lmaydi. Uni avval bo'laklarga (baytlarga) ajratib, ketma-ket yuborib, narigi tomonda yig'ish yo'riqnomasi (kodi) asosida qayta qurish kerak.

Buning 3 ta asosiy ilmiy sababi quyidagilar:

1. Tarmoq faqatgina "1" va "0" larni taniydi (Fizik sabab)

Internet kabellari (optik tola yoki mis simlar) va Wi-Fi to'lqinlari "Python ro'yxati" yoki "Matnli qator" nima ekanligini bilmaydi. Ular faqatgina yorug'lik chaqnashlari yoki elektr signallari (elektr bor = 1, elektr yo'q = 0) orqaligina ma'lumot tashiydi. Shuning uchun kompyuterdan chiqib ketayotgan har qanday narsa oxir-oqibat universal tilga — **baytlar (bytes) oqimiga** aylanishi majburiydir.

2. Dasturlash tillari orasidagi farq (Mantiqiy sabab)

Tasavvur qiling, siz Python dasturidan turib Java yoki C++ da yozilgan serverga xabar jo'natyapsiz.

- Pythondagi `Dictionary` (lug'at) obyekti xotirada qanday saqlanishini faqatgina Python'ning o'zi tushunadi.
- Agar siz bu obyektни xomligicha jo'natsangiz, C++ serveri bu kelgan narsani o'qiy olmaydi (chunki unda Dictionary o'rniga Map degan boshqacha tuzilma bor). **Baytlarga o'girish (Encoding / Serialization)** ma'lumotni hech qaysi tilga tegishli bo'lmagan, barcha uchun tushunarli bo'lgan neytral (xalqaro) formatga tushirib beradi.

3. Kodlash standartlari (Encoding)

Oddiy matnni ("Salom") olaylik. Kompyuterlar harflarni bilmaydi, ular raqamlarni biladi xolos. "S" harfi qaysi raqamga teng ekanligini ikkala kompyuter ham bir xil tushunishi kerak. Shuning uchun matnlar **UTF-8** kabi xalqaro standartlar yordamida baytlarga o‘giriladi (`.encode('utf-8')`).

- Siz "S" ni yuborasiz -> Encode uni `0x53` (bayt) ga aylantiradi.
- Tarmoq `01010011` ni tashiydi.
- Qabul qiluvchi `01010011` ni olib, uni `0x53` deb tushunadi va uni Decode (`.decode('utf-8')`) qilib ekranga yana "S" harfini chiqaradi.

Xulosa: Amaliyotda bu qanday qilinadi?

1. **Oddiy matnlar uchun:** `xabar.encode('utf-8')` qilib baytga (`b"xabar"`) aylantiramiz. Olganda esa `xabar.decode('utf-8')` qilamiz.
2. **Murakkab Python obyektlari uchun (Masalan ro‘yxatlar, lug‘atlar):** Avval ularni oddiy matnga (JSON) o‘giramiz (`json.dumps(obj)`), keyin esa u matnni baytga o‘giramiz. Olganda esa jarayonni teskarisiga bajaramiz.

15-savol: Server dasturini ishga tushirishda IP manzil hamda Port tushunchalarining aniq vazifasi nima va nega serverni lokal tarmoqda barchaga ko‘rinadigan qilish uchun IP manzilni “0.0.0.0” ga sozlash qo‘llaniladi?

(Sizda savollar raqami biroz adashib ketgan ko‘rinadi, xronologiya bo‘yicha buni 15-savol deb belgilab oldim).

Javob: Tarmoq dasturlashda IP manzil va Port tushunchalari server kompyuterining internetdagi (yoki lokal tarmoqdagi) aniq **geografik va jismoniy koordinatalari** hisoblanadi. Ularning vazifasini hayotiy analogiya bilan juda oson tushunish mumkin:

1. IP manzil va Portning aniq vazifasi nima?

Tasavvur qiling, sizga kuryer orqali muhim posilka (tarmoq xabari) kelishi kerak.

- **IP Manzil (Masalan: `192.168.1.15` yoki `142.250.190.46`):** Bu siz yashaydigan **ko‘p qavatli binoning aniq manzili** (Toshkent shahar, Navoiy ko‘chasi, 10-uy). IP manzil kompyuterni (yoki serverni) butun boshli tarmoq ichidan topib borish uchun xizmat qiladi. Kuryer aynan shu IP manzil orqali qaysi kompyuterga borishni biladi.
- **Port (Masalan: `80` , `443` , `8080`):** Bu o‘sha ko‘p qavatli binodagi **sizning kvartirangiz (eshigingiz) raqami**. Bitta kompyuter ichida bir vaqtning o‘zida

ham veb-sayt (80-port), ham ma'lumotlar bazasi (5432-port), ham o'yin serveri (27015-port) ishlashi mumkin. Kuryer binoga kelgach, xatni aynan qaysi dasturga (qaysi xonaga) berish kerakligini Port orqali aniqlaydi.

2. Nega IP manzil "0.0.0.0" ga sozlanadi?

Dasturchilar ko'pincha server kodini yozib ishga tushirayotganda `app.run(host='0.0.0.0', port=8080)` degan qatorni yozishadi. Bu nima degani?

Kompyuterda odatda bitta emas, bir nechta "tarmoq eshiklari" (tarmoq interfeyslari) bo'ladi. Serverni ishga tushirishda IP manzilni qanday tanlash, uning kimlarga xizmat ko'rsatishini belgilab beradi:

- **127.0.0.1 (Localhost - Faqat o'zim uchun):** Agar serverni shu manzil bilan ishga tushirsangiz, kompyuter faqat **o'zining ichidan** kelgan so'rovlarni qabul qiladi. Agar siz telefoningizdan (shu Wi-Fi da bo'lsangiz ham) kompyuterga ulanmoqchi bo'lsangiz, server sizni kiritmaydi. Bu kod yozish va xavfsiz sinovdan o'tkazish uchun kerak.
- **192.168.1.15 (Lokal IP - Faqat bitta eshik):** Agar serverni faqatgina Wi-Fi ga ulangan IP orqali ishga tushirsangiz, u faqatgina Wi-Fi orqali kelganlarni qabul qiladi. Agar kimdir LAN kabeli orqali (boshqa IP bilan) ulansa, ishlamaydi.
- **0.0.0.0 (Barcha eshiklar lang ochiq):** Bu **"Wildcard"** (maxsus belgi) hisoblanadi. U kompyuterga shunday buyruq beradi: *"Menda qanday tarmoq interfeysi bo'lsa (Wi-Fi, kabel, Localhost, VPN) — hammasidan kelayotgan mijozlarni qabul qilaver!"*.

Hayotiy ssenariy: Siz noutbukda yangi veb-sayt (Python+PyQt5 yoki Flask) yozdingiz va uni telefoningizda qanday ko'rinishini sinab ko'rmoqchisiz. Agar serverni `127.0.0.1` da yoqsangiz, telefon uni ko'rmaydi. Agar `0.0.0.0` da yoqsangiz, endi telefon brauzeriga kompyuterning IP manzilini yozish orqali bimalol ulanishingiz va saytni telefoningizda ham ko'rishingiz mumkin bo'ladi.

16-savol: Bitta server dasturiga bir vaqtning o'zida o'nlab mijozlar (client) ulana olishi va ma'lumot almashishi uchun arxitekturaning qanday tashkil etish kerak va bunda qanday muammolar kelib chiqadi?

Javob: Oddiy tarzda yozilgan tayanch server dasturi (sinxron server) bir vaqtning o'zida faqat **bitta** mijozga xizmat ko'rsata oladi. Chunki uning ishlash mantig'i ketma-

ketlikka asoslangan: u bitta mijozni qabul qiladi (`accept()`), uning xabarini kutadi (`recv()`) va javob qaytarmaguncha boshqa mijozlarni eshik tagida (navbatda) kutib turishga majbur qiladi.

Bir vaqtning o'zida o'nlab, yuzlab mijozlarga parallel xizmat ko'rsatish uchun server arxitekturasini tubdan o'zgartirish talab etiladi. Buning asosan ikkita mashhur yondashuvi mavjud:

1. Ko'p oqimli arxitektura (Multi-threading)

Bu an'anaviy va tushunish uchun eng oson yondashuv.

- **Qanday ishlaydi?** Serverning bitta "Asosiy oqimi" (Main Thread) doimiy ravishda eshik og'zida poyloqchilik qiladi (`accept()`). Yangi mijoz kelishi bilan u darhol yangi, mustaqil "Ishchi oqim" (Worker Thread) yaratadi va mijozni o'sha ishchiga topshirib, o'zi yana eshikka qaytadi. Shunday qilib, 50 ta mijoz kelsa, 50 ta mustaqil parallel oqim paydo bo'ladi.
-
- **Qachon ishlatiladi?** Har bir mijoz bilan aloqa uzoq vaqt davom etadigan va uzilmaydigan jarayonlarda (masalan, fayl yuklab olish yoki jonli chatlar). Python'da bu `threading` moduli orqali amalga oshiriladi.

2. Asinxron voqealar sikli (Asynchronous I/O)

Bu zamonaviy, o'ta yuqori unumdorlikka ega yondashuv (Node.js, Nginx va Python'ning `asyncio` moduli shu asosda ishlaydi).

- **Qanday ishlaydi?** Bu yerda o'nlab oqimlar yaratilmaydi. Faqat bitta oqim va bitta "Voqealar sikli" (Event Loop) bo'ladi. Server mijozdan xabar kutayotganda qotib turmaydi, balki darhol boshqa mijozning ishiga o'tib ketadi. Ya'ni server o'z vaqtini mijozlar o'rtasida millisoniyalarda taqsimlab, xuddi hammaga bir vaqtda xizmat ko'rsatayotgandek tasavvur uyg'otadi.
-
- **Qachon ishlatiladi?** Bir vaqtning o'zida 10,000 lab mijozlar ulanishi kerak bo'lgan, lekin xabarlar juda tez va qisqa uzatiladigan tizimlarda (masalan, Telegram xabarlari, onlayn o'yin signallari).

Kelib chiqadigan xavf va muammolar

Parallel arxitekturani qurish o'ziga yarasha juda murakkab "bosh og'riqlarni" keltirib chiqaradi:

A. Ma'lumotlar to'qnashuvi (Race Conditions)

Aytaylik, serverdagi bitta bank hisobida 100 ming so‘m pul bor. Ikkita turli joydan (ikkita parallel thread'dan) bir vaqtning o‘zida mijozlar 100 ming so‘mni yechib olish tugmasini bosdi. Ikkala thread ham bir xil millisoniyada hisobni tekshiradi (pul yetarli!) va yechib oladi. Natijada mijoz 200 ming so‘m yechib oladi, bazada esa xatolik yuz beradi.

- **Yechimi:** Bitta obyektни bir nechta oqim o‘zgartirmoqchi bo‘lsa, "Qulflar" (Locks / Mutex) ishlatish kerak. Ya'ni birinchi mijoz hisobga kirdimi, baza qulflanadi, ikkinchi mijoz u chiqib ketmaguncha kutib turadi.

B. Resurslarning tugab qolishi (Thread Exhaustion)

Har bir yangi Thread yaratish kompyuterning operativ xotirasidan (RAM) ma'lum joy va protsessor vaqtini oladi. Agar serveringizga bir vaqtda 5000 ta mijoz ulansa va siz 5000 ta thread yaratsangiz, operatsion tizim ularni boshqara olmay (Context Switching qilaverib) qulab tushadi (Crash).

- **Yechimi:** Har bir mijoz uchun yangi oqim yaratmasdan, "Thread Pool" (Oqimlar hovuzi) ishlatish. Masalan, oldindan qat'iy qilib 100 ta ishchi oqim yaratib qo‘yiladi. 5000 ta mijoz kelsa ham, shu 100 ta ishchi ularga navbat bilan xizmat ko‘rsatadi.

C. Qotib qolish (Deadlocks)

Bu holat ikkita parallel jarayon bir-birini cheksiz kutib qolganda yuz beradi. Masalan: A mijoz B mijozdan pul o‘tkazmasini kutayotgani uchun tranzaksiyani ochiq ushlab turibdi, B mijoz esa A mijoz o‘zining tizimidan chiqishini kutib turibdi. Ikkisi ham bir-biriga yo‘l bermaydi va butun server qotib qoladi.

17-savol: Soketlar bilan ishlashda bloklanuvchi (blocking) rejim va kutib turmaydigan (non-blocking) rejim dasturning ishlash mantig‘iga va tezligiga qanday ta’sir ko‘rsatadi?

Javob: Soketlardagi "Bloklanuvchi" (Blocking) va "Kutib turmaydigan" (Non-blocking) rejimlar dasturning tarmoqqa nisbatan naqadar "**sabrli**" yoki "**shoshqaloq**" ekanligini belgilab beruvchi eng asosiy tushunchalardir.

Ular dastur logikasi va tezligini mutlaqo ikki xil o‘zanga burib yuboradi. Keling, ularni hayotiy misol — "**Pochtalyonni kutish**" orqali tushunamiz:

1. Bloklanuvchi rejim (Blocking Mode) – "Sabrli kutish"

Python'da soket yaratganingizda u sukut bo‘yicha (default) aynan shu rejimda ishlaydi.

- **Qanday ishlaydi?** Siz dasturda `mijoz, manzil = server.accept()` (yangi mijozni kutish) yoki `ma_lumot = mijoz.recv(1024)` (xabar kelishini kutish) deb yozsangiz, dastur aynan shu qatorga kelganda **to'xtaydi (muzlaydi)**. Toki tarmoqdan haqiqatdan ham kimdir ulanmaguncha yoki xabar yubormaguncha, dastur keyingi qatorga o'tmaydi. Xuddi pochta yon kelmaguncha eshik tagida qimirlamay kutib o'tirgandek.
- **Mantiqqa ta'siri:** Kodni yozish va o'qish juda oson (ketma-ketlik chiroyli). Lekin, bitta mijozdan xabar kutayotganda dastur qotib qolgani uchun, u boshqa mijozlarga xizmat ko'rsata olmaydi. Agar GUI (oyna) dastur bo'lsa, oyna "Not Responding" bo'lib oqarib qoladi.
- **Tezlikka ta'siri:** Bitta mijoz bilan ishlashda yaxshi, lekin ko'p mijozlarga xizmat ko'rsatish kerak bo'lganda (agar Multi-threading ishlatilmasa) tezlik falaj bo'ladi.

2. Kutib turmaydigan rejim (Non-blocking Mode) – "Shoshqaloq tekshirish"

Bu rejimni yoqish uchun dasturda maxsus `socket.setblocking(False)` buyrug'i yoziladi.

- **Qanday ishlaydi?** Dastur `recv()` ga kelganda tarmoqqa qaraydi: ma'lumot bormi? Bor bo'lsa darhol oladi. **Yo'q bo'lsa, umuman kutib o'tirmaydi!** U darhol operatsion tizimdan "Hozircha hech narsa yo'q" degan xatoni (`BlockingIOError`) oladi va darhol keyingi qatorga o'tib ketib, boshqa ishlarni qilaveradi. Xuddi pochta pochta qutisidan tekshirib, bo'sh bo'lsa darhol uy ishlarini qilishda davom etgan kabi.
- **Mantiqqa ta'siri:** Kod ancha murakkablashadi. Siz tinimsiz ravishda xatoliklarni ushlab turuvchi `try/except` yoki maxsus `select / epoll` modullarini ishlatib, cheksiz sikl ichida barcha mijozlarni bitta-bitta aylanib tekshirib (Polling) chiqishingiz kerak bo'ladi.
- **Tezlikka ta'siri:** Bu rejim orqali yozilgan serverlar **juda tez va yengil** bo'ladi. Bitta oqimning o'zi minglab mijozlardan bir soniya ichida xabar kelgan-kelmaganini so'rab chiqishga ulguradi va xotira (RAM) tejab qolinadi. Zamonaviy `asyncio` kutubxonalari aynan shu rejim ustiga qurilgan.

Qisqacha Xulosa

Xususiyat	Blocking (Bloklanuvchi)	Non-blocking (Kutib turmaydigan)
Harakat	Xabar kelmaguncha dastur qotib turadi	Xabar yo‘qmi? Darhol boshqa ishga o‘tadi
Kod murakkabligi	Oson (Ketma-ket to‘g‘ri oqim)	Murakkab (Doimiy aylanma tekshiruv, try/except)
Resurs talabi	Ko‘p (Ko‘p mijoz uchun ko‘p Thread kerak)	Kam (Bitta Thread hammasiga ulguradi)
Qayerda ishlatiladi?	Kichik, oddiy skriptlarda va ko‘p oqimli (Multi-thread) serverlarda	Yirik, minglab mijozli serverlarda (Node.js, Nginx, Asyncio)

18-savol: Internet yoki tarmoq uzilib qolishi, yoxud mijozning birdan javob bermay qo‘yishi (timeout) kabi kutilmagan favqulodda holatlarda dastur “qulab tushmasligi” uchun qanday ehtiyot choralari ko‘riladi?

Javob: Tarmoq dasturlashdagi eng katta va eng xavfli xato — bu **"Internet doim barqaror ishlaydi"** deb o‘ylashdir. Aslida tarmoq har doim ishonchsiz: kabel uzilib ketishi mumkin, Wi-Fi o‘chib qolishi mumkin yoki mijozning kompyuteri to‘satdan qotib qolishi (yoki tokdan uzilishi) mumkin.

Bunday holatlarda dastur qulab tushmasligi (Crash bo‘lmasligi) va o‘zini tiklay olishi (Resilience) uchun quyidagi 4 ta asosiy "himoya yostiqlari" qo‘llaniladi:

1. Xatoliklarni ushlab qolish (Try/Except bloklari)

Bu eng birinchi va eng muhim qadam. Tarmoqqa tegishli har qanday operatsiya (`connect`, `send`, `recv`) albatta `try...except` bloki ichiga olinishi shart.

- Agar dastur tarmoq uzilgan vaqtda nimanidir jo‘natmoqchi bo‘lsa, `OperationError` Tizim darhol xatolik otadi (`ConnectionResetError`, `BrokenPipeError`).
- Agar `try/except` bo‘lmasa, butun dasturingiz (va uning oynasi) yopilib ketadi.
- Bu blok orqali biz xatoni jimgina "yutib yuboramiz" va dasturga: *"Hechqisi yo‘q, tarmoq uzilibdi, ishlashda davom etaver"* deb buyruq beramiz.

2. Kutish vaqtini chegaralash (Timeouts)

Tasavvur qiling, server mijozga fayl yubordi va undan "Oldim" degan javobni kutmoqda (`recv()`). Aynan shu soniyada mijozning chirog'i o'chib qoldi. Mijoz kompyuteri aloqani rasman uzishga ulguradi. Buni **"Zombi ulanish" (Half-open connection)** deyiladi.

- Agar dastur bloklanuvchi (blocking) rejimda bo'lsa, u cheksiz muddatgacha — soatlab, kunlab o'sha mijozdan javob kutib qolib qolaveradi.
- **Yechim:** `socket.setdefaulttimeout(5.0)` buyrug'i beriladi. Bu dasturga: *"Faqat 5 soniya kut, agar mijozdan sado chiqmasa, uni 'o'lik' deb hisobla va ulanishni majburan uzib tashla"* degan buyruqdir. Bu tizimni qotib qolishdan asraydi.

3. "Tomir urishi" (Heartbeat / Keep-Alive) mexanizmi

Ba'zida tarmoq ulangan, lekin internet ishlamayotgan bo'lishi mumkin. Dastur buni darhol seza olmaydi.

- **Yechim:** Ikkala tomon (Server va Mijoz) o'zaro kelishib oladi va fonda har 10 soniyada bir-biriga bo'm-bo'sh, juda kichik paket: *"Men tirikman (Ping)"* deb yuborib turadi. Ikkinchi tomon bunga *"Men ham (Pong)"* deb javob qaytaradi.
- Agar ketma-ket 3 ta "Ping" ga "Pong" javobi kelmasa, dastur aloqa uzilganini o'zi tushunib yetadi va foydalanuvchiga xabar chiqaradi.

4. Avtomatik qayta ulanish va Backoff (Auto-Reconnect)

Dastur aloqa uzilganini tushunib yetgach, vahimaga tushib yopilib ketmaydi. U o'zini **"Kutish rejimiga"** o'tkazadi va orqa fonda serverga qayta ulanishga harakat qila boshlaydi.

- **Exponential Backoff (o'sib boruvchi kutish):** Serverni tinimsiz so'rovlar bilan qiynamaslik uchun aqlli dasturlar qayta ulanish oralig'ini oshirib boradi: avval 1 soniya kutadi, o'xshamasa 2 soniya, keyin 4 soniya, 8 soniya va hokazo. Internet qaytib kelishi bilan dastur hech narsa ko'rmagandek o'z ishini davom ettirib ketaveradi.

19-savol: Tarmoq orqali ochiq ma'lumot va xabarlar almashish foydalanuvchilar uchun qanday xavf tug'diradi va kelajakda Python socketlariga SSL/TLS kabi xavfsizlik protokollarini ulash nima maqsadda qilinadi?

Javob: Oddiy TCP yoki UDP socketlar orqali yuborilgan ma'lumotlar tarmoqda **ochiq matn (plain text)** ko'rinishida sayohat qiladi. Bu degani, siz "Parolim: 12345" deb

yuborsangiz, u kabeldan aynan shu harflar ko‘rinishida o‘tadi. Bu esa xavfsizlik nuqtai nazaridan halokatli hisoblanadi.

1. Ochiq ma'lumot almashish qanday xavflarni tug‘diradi?

Tarmoqdagi bu zaiflik asosan **"o‘rtadagi Odam" (Man-in-the-Middle - MitM)** hujumlariga yo‘l ochib beradi.

- **Tutib olish (Packet Sniffing):** Siz kafelardagi bepul Wi-Fi dan foydalanayotganingizda, o‘sha tarmoqqa ulangan har qanday havaskor xaker "Wireshark" kabi maxsus dasturlar yordamida sizning kompyuteringizdan ruterga qarab ketayotgan barcha xatlarni havoda tutib olishi va o‘qishi mumkin. Sizning yozishmalaringiz, bank kartalaringiz, parollaringiz ochiq-oydin ko‘rinib turadi.
- **Ma'lumotni o‘zgartirish (Tampering):** Xaker nafaqat xatni o‘qishi, balki uni o‘zgartirib yuborishi ham mumkin. Masalan, siz serverga "Akamning hisobiga 100 o‘tkaz" degan xat yubordingiz. Xaker havoda uni ushlab, "Xakerning hisobiga o‘tkaz" deb tahrirlaydi va serverga yuboradi. Server esa buni siz yubordingiz deb o‘ylaydi.
- **Soxta Server (Spoofing):** Xaker o‘zini siz ulanmoqchi bo‘lgan haqiqiy server (masalan, Google) qilib ko‘rsatishi va barcha ma'lumotlaringizni o‘ziga yig‘ib olishi mumkin.

2. SSL/TLS protokollarini ulash nima maqsadda qilinadi?

Yuqoridagi dahshatli muammolarni hal qilish uchun **SSL (Secure Sockets Layer)** va uning zamonaviy avlodi **TLS (Transport Layer Security)** o‘ylab topilgan. Biz har kuni ko‘radigan brauzerdagi qulf belgisi (`https://`) aynan shu texnologiya ishlashini bildiradi.

Python'da yozgan oddiy soketingizni `ssl.wrap_socket()` funksiyasi orqali SSL qobig‘iga o‘rasangiz, u quyidagi uchta mo‘jizani yaratadi:

1. **Harbiy darajadagi Shifrlash (Encryption):** Siz yuborgan "Parolim: 12345" xati kompyuteringizdan chiqishidanoq o‘qib bo‘lmaydigan abrakadabraga (masalan, `x8f@!z9Q...`) aylanadi. Xaker uni ushlab olsa ham, kalitsiz bu matnni yechish uchun dunyodagi eng kuchli superkompyuterga minglab yillar kerak bo‘ladi.
2. **Yaxlitlik (Data Integrity):** SSL har bir paketga raqamli muhr (MAC) bosadi. Agar xaker yo‘lda xatning bitta harfini o‘zgartirsa ham, muhr buziladi va Server bu xatni qabul qilmay, darhol aloqani uzadi.

3. **Haqiqiylikni tasdiqlash (Authentication):** Server mijozga o‘zining "Raqamli Sertifikati"ni (Pasportini) ko‘rsatadi. Bu sertifikat ishonchli xalqaro tashkilotlar (CA) tomonidan tasdiqlangan bo‘ladi. Mijoz bu orqali rostdan ham xakerning soxta serveriga emas, balki aynan haqiqiy manzilga ulanganiga 100% ishonch hosil qiladi.

20-savol: Eng katta muammolardan biri sifatida, tarmoqdan ma’lumot kutib turgan funksiyalarni PyQt5 ning vizual interfeysi bilan qanday qilib shunday birlashtirish kerakki, interfeys hech qachon qotib qolmasin?

Javob: Bu muammo (GUI qotib qolishi) har bir boshlovchi dasturchi to‘qnash keladigan eng katta to‘siqdir. Muammoning ildizi shundaki: **PyQt5 ning oynasi (Event Loop) ham, tarmoqdan xabar kutuvchi `recv()` funksiyasi ham "Bloklanuvchi" (kutib turuvchi) tabiatga ega.** Agar ularni bitta joyga (Asosiy oqimga) qamab qo‘ysangiz, ular bir-birini falaj qilib qo‘yadi.

Interfeys **hech qachon qotib qolmasligi** uchun, arxitekturani **"QThread + pyqtSignal"** qolipida, qat’iy uchta qoidaga amal qilgan holda qurish kerak:

1-qoida: Qora ishlarni orqa fonga suring (QThread)

Hech qachon `socket.recv()`, `requests.get()` yoki uzoq davom etuvchi bazaga ulanish jarayonlarini asosiy oynangizdagi tugma bosilganda ishlaydigan funksiya ichiga yozmang. Buning o‘rniga, tarmoq bilan gaplashadigan barcha kodlarni `QThread` dan meros olgan mutlaqo alohida sinf (class) ning `run()` metodi ichiga yashiring. Bu "Ishchi oqim" (Worker Thread) deb ataladi.

2-qoida: Qat'iy chegara (Asosiy oqimga teginmang)

Orqa fondagi Ishchi oqim (QThread) tarmoqdan ma'lumotni qabul qilib olgach, u to‘g‘ridan-to‘g‘ri ekrandagi oynaga yozishga haqqi yo‘q. Ya'ni, QThread ichidan turib `self.label.setText("Xabar keldi")` deyish qat'iyani taqiqlanadi. Bu xotira to‘qnashuviga (Crash) olib keladi. Interfeys elementlariga faqatgina Asosiy oqim (Main Thread) tegishi mumkin.

3-qoida: Xavfsiz ko‘prik (pyqtSignal)

Ishchi oqim ma'lumotni oldi, lekin oynaga yozolmaydi. Unda nima qiladi? U PyQt5 ning maxsus xabarchisi — `pyqtSignal` dan foydalanadi. Ishchi oqim ma'lumotni signalga o‘raydi va havoga otadi (`emit()`). Asosiy oqimdagi interfeys esa o‘sha signalni o‘zining tayyor xaltachasi (`Slot`) orqali tutib oladi va oynani **o‘zi** yangilaydi.

Arxitekturani kodda ko‘ramiz (Oltin Qolip)

Quyidagi kod har qanday og‘ir tarmoq dasturida GUI qotmasligi uchun andoza (shablon) hisoblanadi:

```
from PyQt5.QtCore import QThread, pyqtSignal
from PyQt5.QtWidgets import QApplication, QWidget, QVBoxLayout, QPushButton, QLabel
import time, sys

# 1. TARMOQ ISHCHISI (Orqa fon)
class TarmoqOqimi(QThread):
    # Asosiy oqimga matn yuborish uchun signal yaratamiz
    xabar_signali = pyqtSignal(str)

    def run(self):
        # Bu yerda tarmoqdan javob kutish jarayoni ketadi
        time.sleep(3) # Masalan, 3 soniya kutilmoqda (Blocking!)
        yangi_xabar = "Serverdan javob keldi: 200 OK!"

        # 2. To‘g‘ridan-to‘g‘ri UI ga yozmasdan, signal orqali yuboramiz
        self.xabar_signali.emit(yangi_xabar)

# 3. ASOSIY OYNA (Interfeys)
class AsosiyOyna(QWidget):
    def __init__(self):
        super().__init__()
        self.layout = QVBoxLayout(self)

        self.label = QLabel("Kutmoqda...")
        self.tugma = QPushButton("Tarmoqqa ulanish")
        self.tugma.clicked.connect(self.tarmoqni_boshlash)

        self.layout.addWidget(self.label)
        self.layout.addWidget(self.tugma)

        # Oqimni tayyorlab qo‘yamiz
        self.ishchi = TarmoqOqimi()
        # Ishchidan kelgan signalni ekranni yangilovchi slotga ulaymiz
        self.ishchi.xabar_signali.connect(self.ekranni_yangila)

    def tarmoqni_boshlash(self):
        self.label.setText("Yuklanmoqda (oina qotgani yo‘q!)...")
        self.ishchi.start() # Ishchini ishga tushiramiz

    def ekranni_yangila(self, xabar):
        self.label.setText(xabar) # Kelgan xabarni ekranga yozamiz

app = QApplication(sys.argv)
```

```
oyna = AsosiyOyna()  
oyna.show()  
sys.exit(app.exec_())
```